TD N°1 - Commandes de base : manipulation de fichiers, répertoires et liens

Objectif : Se familiariser avec les commandes de gestion de répertoires et fichiers.

Notions : hiérarchie/arborescence, répertoire personnel/d'accueil (*home directory*), répertoire courant de travail (*current working directory*), fichiers et répertoires, chemin d'accès (absolu/relatif).

Notations utiles :

- pour un chemin d'accès (avec les commandes **cd** ou **ls** par ex.) :
  - `.` répertoire courant
  - `..` répertoire parent
  - `~` répertoire d'accueil
  - `~-` (ou `-` , mais avec **cd** seulement) : répertoire courant de travail précédent
- caractère générique/*joker* :
  - `*` qui désigne tous les objets du dossier courant (sauf les fichiers « *cachés* » , désignés par `.*`)

RAPPEL : La commande **man [commande]** (pour *manual*) donne une aide pour chaque commande. Une fois que la page manuel est affichée, les touches de déplacement permettent de naviguer dedans. Pour quitter et revenir à la ligne de commande, il suffit de taper **Q**. Il y a beaucoup de subtilités que l'on passe ici sous silence, n'hésitez pas à vous servir de `man` pour en apprendre plus !

Rappel : Manipulation de **fichiers** et **répertoires**

Action	(sur) **Fichiers**	(sur) **Répertoires**
Afficher contenu	`cat` conCATenate `more`, `less`	`ls` ⇒ voir (1) LiSt options `-a`, `-A` , `-l`, `-n`, `-i`, `-s`, `-R`
Afficher propriétés (détails comme nom du propriétaire, date de mise à jour, permissions ...)	`ls` **-l** (ou `ls` **-n**) ⇒ voir (1) options `-i`, `-s` , `-h`	`ls` **-dl** (ou `ls` **-dn**) `-d` pour empêcher le « développement » du contenu du répertoire options `-i`, `-s`
Copier	`cp` CoPy option `-i`	`cp` (-r, -R)
Changer de nom (renommer) ou : Déplacer ⇒ voir (3)	`mv` MoVe option `-i`	`mv`
Changer de répertoire		`cd` Change Directory
Créer	⇒ voir (2)	`mkdir` MaKe DIRectory option `-p` pour créer les parents (ascendants) si nécessaire
Détruire (Supprimer)	`rm` ReMove	`rmdir` ReMove DIRectory (si répertoire vide !) ou `rm -r` (**prudence** !)
Afficher répertoire courant		pwd Print Working Directory
Créer lien (direct ou symbolique)	`ln` LiNk sans option : lien direct ⇒ une **référence** (ou « **étiquette** ») supplémentaire pour désigner le fichier option `-s` pour lien symbolique (« **raccourci** »)	`ln -s` lien symbolique lien direct (hard link) pas possible !

(1) s'il n'y a pas d'argument, la commande `ls` liste le contenu du répertoire courant.
(2) pour créer un fichier vide, on peut utiliser `touch` *fichier*, dont le rôle principal est de modifier

l'horodatage d'un fichier, ou la redirection >*fichier*, qui sera étudiée ultérieurement.
(3) il s'agit de déplacement dans la hiérarchie (déplacement *logique*) ; il n'y a de réel déplacement *physique* (sur le disque) que si les objets « source » et « cible » de la commande `mv` sont dans des systèmes de fichiers différents.

Quelques précisions pour `cp` et `mv` :

- avec des **fichiers ordinaires** (réguliers)

  `cp` *fichier1 fichier2* crée une copie de *fichier1*, nommée *fichier2*

  `cp` *fichier ... repertoire* le dossier de destination *repertoire* doit exister ; il est possible d'indiquer plusieurs fichiers « source »
  les copies, dans *repertoire*, ont le même nom que l'original.

  `mv` *fichier1 fichier2* renomme *fichier1* en *fichier2*

  `mv` *fichier ... repertoire* déplacement vers *repertoire*, qui doit exister.
  il est possible d'indiquer plusieurs fichiers.
  les fichiers déplacés gardent leur nom d'origine sous *repertoire*.

  Attention : Avec `cp` ou `mv` : **Risque** d'écraser des fichiers ayant le même nom, s'il existe déjà !
  Utilisez l'option **-i** pour protection (avoir un message avertissant d'un risque d'écrasement).

- avec des **répertoires**

  `cp -r` *repertoire1 repertoire2* copie récursive de toute l'arborescence *repertoire1* vers *repertoire2*

  différence selon que *repertoire2* existe ou non :

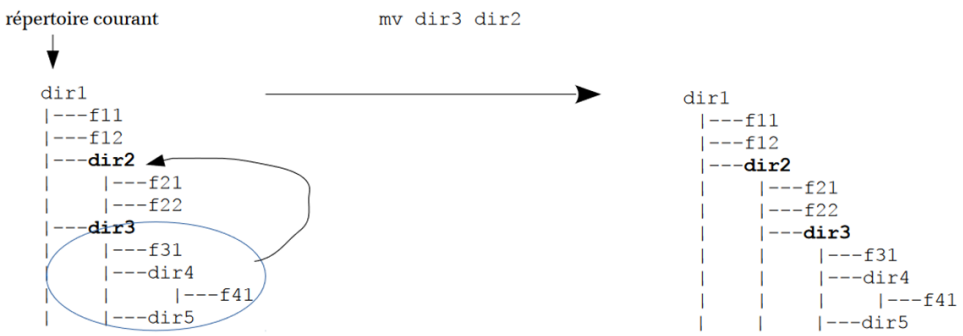
  - si le dossier cible *repertoire2* **existe**, copie de toute l'arborescence *repertoire1* vers *repertoire2* (c.a.d., **y compris** la « racine » *repertoire1*)
  - si *repertoire2* **n'existe pas**, copie de tous les objets situés sous *repertoire1* vers *repertoire2*, qui est automatiquement créé (autrement dit : les **descendants** de *repertoire1* sont copiés sous *repertoire2*, nouvellement créé).

  ⇒ équivalent dans ce cas à : `mkdir repertoire2 ; cp -r repertoire1/. repertoire2/`
  Remarque : on utilise ici `repertoire1/.` et non `repertoire1/`$\star$, car le joker $\star$ ne sélectionne pas les fichiers cachés (ceux dont le nom commence par un point).

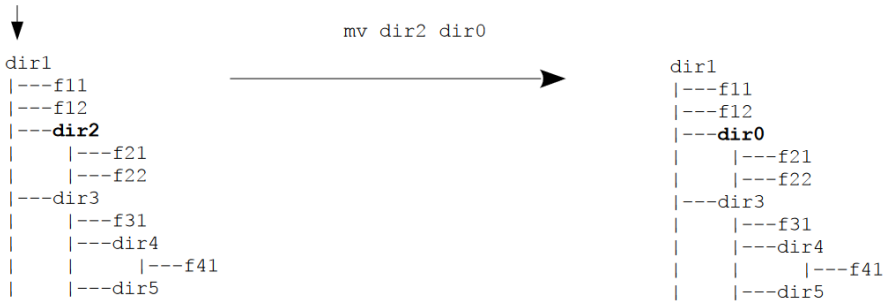
  `mv` *repertoire1 repertoire2* déplacement du « sous-arbre » *repertoire1* ou renommage selon le cas

Attention : comme avec `cp`, il y a une différence selon que le répertoire « cible » *repertoire2* existe ou non.

Exemples :



NB : Les `f`$\star$ sont des fichiers (ordinaires)
Le répertoire cible *dir2* existant, tout l'arbre *dir3* (racine *dir3* comprise) est déplacé en-dessous de *dir2*.



Dans cet exemple, le répertoire cible *dir0* n'existe pas. On constate que *dir2* est (simplement) renommé en *dir0*.

Des archives contenant les arborescences initiales sont néanmoins disponibles dans le dossier `Ressources_TD1` sur *Moodle*. Elles vous permettent de reproduire les manipulations sur votre machine ou de recommencer un exercice depuis son état initial.

Pour décompresser une archive dans un dossier de test :

```
mkdir -p "$HOME/S11/TD1"
unzip nom_archive.zip -d "$HOME/S11/TD1/nom_exercice"
```

Ne décompressez pas les archives directement dans votre répertoire d'accueil afin de ne pas mélanger les fichiers du TD avec vos documents personnels.

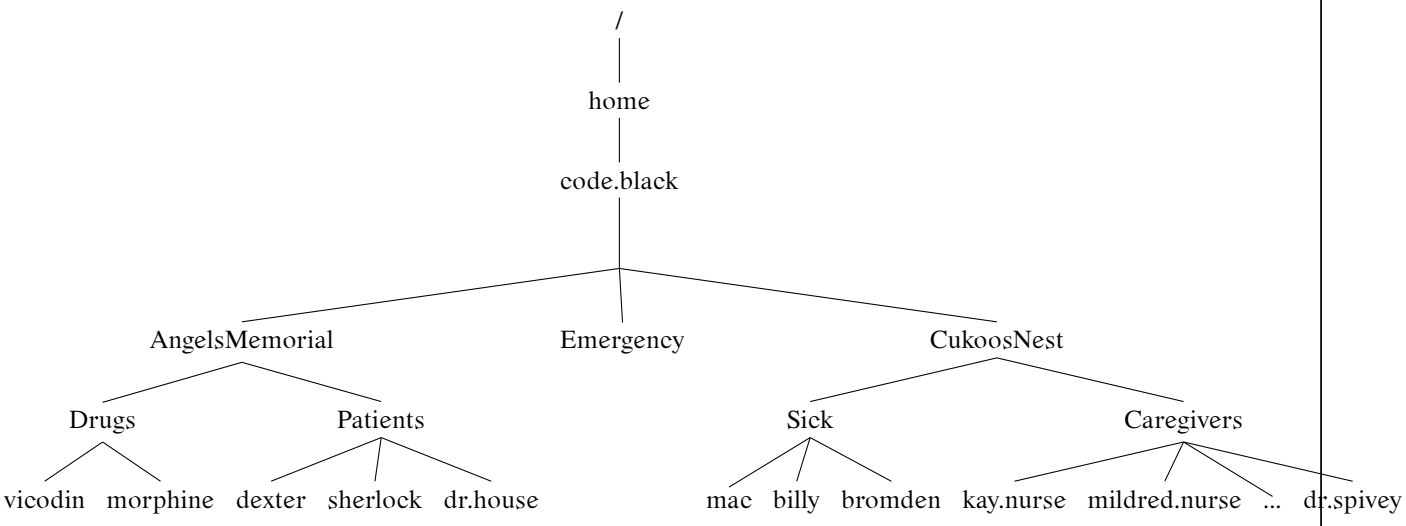
Exercices

Exercice 1

On part de la même arborescence que dans les deux exemples précédents ; on suppose que le répertoire courant est *dir1*.

1. Dessinez l'arborescence résultant de la commande : `mv dir3/dir4 dir4`

Exercice 2



Au départ, le **répertoire courant** est */home/code.black*. C'est aussi votre répertoire d'accueil lors de la connexion.

Dans la série médicale Code Black, les urgences de l'Angels Memorial Hospital sont les plus surpeuplées de la nation. Un « code noir » se produit quand il y a plus de patients que de ressources…

On vous demande :

1. a) De créer l'arborescence des répertoires **sans** utiliser l'option **-p**
   Indice : vous devez créer un répertoire parent avant de créer ses descendants.
   b) De créer l'arborescence des répertoires en utilisant l'option **-p**.
   Indice : **-p** permet de créer plus facilement toute une « branche » d'arbre (c.a.d. un répertoire et tous ses ancêtres).
2. Quelle commande permet de connaître le chemin du répertoire courant ?
3. a) Quel est le chemin d'accès absolu du fichier *vicodin* ?
   b) Quel est le chemin d'accès relatif du fichier *vicodin* au répertoire d'accueil ?
4. a) De donner le chemin relatif de *dexter* par rapport à *Drugs* ?
   b) Et son chemin relatif par rapport à *Sick* ?
5. D'écrire la commande pour renommer *sherlock* en *holmes* (**sans changer de répertoire**).
6. D'afficher (**sans changer de répertoire**) les contenus de *vicodin* et *morphine*, l'un à la suite de l'autre (**concaténation**).
7. De descendre dans le répertoire *CukoosNest* et copier le fichier *billy* de *Sick* vers le dossier *Emergency/CookCounty*, **que vous créerez au préalable**.
8. De revenir dans le dossier d'accueil (*code.black*) et de supprimer le fichier *billy* original (celui de *Sick*) - **sans se déplacer au répertoire *Sick***.
9. D'aller dans le répertoire *AngelsMemorial* et d'y déplacer (en gardant le même nom) toutes les infirmières de *Caregivers* (sous *CukoosNest*), c.a.d. les fichiers dont le nom se termine par *.nurse*.
   Rem : Dans *Caregivers*, il peut y avoir d'autres infirmières que celles de l'arborescence ci-dessus, et du personnel soignant autre (médecin, etc)
   Indication : utilisez le caractère (**joker**) ⋆
10. De revenir dans le dossier d'accueil.
11. De copier (pour sauvegarde) tous les fichiers de *Sick* vers *Emergency/CookCounty*.
12. De lister les noms et propriétés des fichiers subordonnés à Patients qui ont un caractère **r** dans leur nom.
13. Quelle est la commande pour lister le contenu (**à un seul niveau**) du répertoire *AngelsMemorial* (**avec les détails des objets trouvés**), **fichiers cachés inclus** ?
    La même question en scrutant toute l'arborescence en-dessous de *AngelsMemorial*.
14. Quelle est la commande pour connaître les propriétés (**pas le contenu !**) du répertoire *Emergency* ?
15. De créer un lien (**non symbolique**) du fichier *dexter* dans le dossier *Sick*.
16. De supprimer le répertoire *Sick* (**et son contenu**).

Exercice 3

Le but ici est de bien comprendre la différence entre lien **direct** et lien **symbolique** et la notion d'**inode** (i-nœud).
Étudiez ce qui suit et tirez-en vos conclusions (si vous avez l'impression d'être dans un « sac de nœuds », l'enseignant/secouriste vous aidera à en sortir !

```
[but@localhost Jungle]$ pwd
/home/but/Tests/Jungle
[but@localhost Jungle]$ ls -ali
163717 drwxr-xr-x 2 but but 4096 2021-08-19 16:00 ./     # [1]
163687 drwxr-xr-x 3 but but 4096 2021-08-19 15:59 ../    # [2]
163722 -rw-r--r-- 1 but but 28 2021-08-19 16:00 tarzan
[but@localhost Jungle]$ ls -ali ..
163687 drwxr-xr-x 3 but but 4096 2021-08-19 15:59 ./     # même n° d'inode qu'en ligne [2] ! pourquoi ?
146882 drwx--x--x 36 but but 4096 2021-08-19 15:56 ../
163717 drwxr-xr-x 2 but but 4096 2021-08-19 16:00 Jungle/# même n° d'inode qu'en ligne [1] ! pourquoi ?
[but@localhost Jungle]$ ln tarzan tarzoon
[but@localhost Jungle]$ ls -li tarzoon tarzan
163722 -rw-r--r-- 2 but but 28 2021-08-19 16:00 tarzoon # même n° d'inode que tarzan
163722 -rw-r--r-- 2 but but 28 2021-08-19 16:00 tarzan  # rem. : le nbre de liens (références) est passé à 2
[but@localhost Jungle]$ ln -s tarzan kingkong
[but@localhost Jungle]$ ls -li tarzan tarzoon kingkong
163722 -rw-r--r-- 2 but but 28 2021-08-19 16:00 tarzoon
163727 lrwxrwxrwx 1 but but 6 2021-08-19 16:10 kingkong -> tarzan
163722 -rw-r--r-- 2 but but 28 2021-08-19 16:00 tarzan
[but@localhost Jungle]$ cat tarzan
je suis le roi de la jungle
[but@localhost Jungle]$ cat tarzoon
je suis le roi de la jungle
[but@localhost Jungle]$ cat kingkong
je suis le roi de la jungle
[but@localhost Jungle]$ rm tarzan
[but@localhost Jungle]$ cat tarzan
cat: tarzan: Aucun fichier ou répertoire de ce type
[but@localhost Jungle]$ cat tarzoon
je suis le roi de la jungle
[but@localhost Jungle]$ cat kingkong
cat: kingkong: Aucun fichier ou répertoire de ce type # tiens ?? ... et pourtant kingkong existe !
[but@localhost Jungle]$ ls -li                        # la preuve ...
163722 -rw-r--r-- 1 but but 28 2021-08-19 16:00 tarzoon
163727 lrwxrwxrwx 1 but but 6 2021-08-19 16:10 kingkong -> tarzan
```

Questions :

1. Pourquoi le **numéro d'inode** affiché pour `.` (ligne [1], depuis *Jungle*) est-il le même que celui affiché pour `Jungle/` (depuis *Tests*) ? Même question pour `..` (ligne [2]) et `.` (depuis *Tests*).
2. Après `ln tarzan tarzoon`, *tarzan* et *tarzoon* partagent le même numéro d'inode et le compteur de liens passe à **2**. Que représente concrètement ce compteur ?
3. Quelle est la différence, du point de vue du système de fichiers, entre le lien créé par `ln tarzan tarzoon` (lien direct) et celui créé par `ln -s tarzan kingkong` (lien symbolique) ?
4. Après `rm tarzan`, pourquoi `cat tarzoon` fonctionne-t-il encore, alors que `cat tarzan` et `cat kingkong` échouent tous les deux ? *tarzoon* et *kingkong* sont-ils dans la même situation vis-à-vis de *tarzan* ?
5. Le dernier `ls -li` montre que *kingkong* existe toujours, pourtant `cat kingkong` échoue. Expliquez cette situation en une phrase.